

iCampus · 智慧校园生活 App 开发文档

版本: 2.0** 编写日期**2026年5月

适用课程: 跨平台移动应用开发 完成人 谈天

学号: 2505224130

一、项目概述

1.1 项目背景

当前校园生活中, 学生获取教务通知、预约校园服务、接收消息提醒等需求分散在微信群、QQ 群、校园官网等多个平台, 存在信息获取不及时、服务流程不统一、操作繁琐等问题。本项目旨在开发一个统一的智慧校园生活移动应用, 整合校园各类信息与服务, 为学生提供一站式的校园生活体验。

1.2 项目目标

本项目为课程期末作业, 目标是实现一个功能完整、运行稳定的基础版智慧校园 App, 具体包括:

- 实现基于学号 / 手机号的用戶身份认证系统
- 完成公告查看、校园服务预约、个人信息管理三大核心功能
- 采用前后端分离架构, 后端服务连接本地 MySQL 数据库进行数据持久化存储
- 基于 UniApp-X 框架开发, 使用 Vue 3 + TypeScript + UTS 语言, 支持 H5 端运行
- 代码结构清晰, 注释规范, 符合软件工程开发标准

1.3 功能范围

本项目仅实现基础核心功能, 不包含复杂的高级功能, 具体功能清单如下:

表格

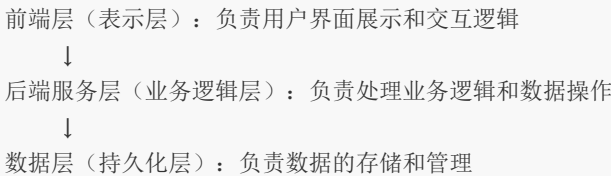
模块	已实现功能	未实现功能（可选扩展）
账号模块	学号 / 手机号 + 密码登录、退出登录、登录状态保持、学号/手机号自动识别	密码重置、记住登录、第三方登录
首页模块	个性化欢迎语、快捷功能入口、最新公告预览	最新消息预览、轮播图、下拉刷新
公告模块	公告列表展示、按类型筛选、公告详情查看	公告搜索、收藏、分享
服务模块	服务类型列表、提交预约申请、我的预约查询、预约状态管理（待处理/已通过/已完成/已取消）、时间沙漏倒计时	服务评价
二手平台	商品列表展示、商品分类、商品详情、发布商品、购买商品、订单管理、订单超时自动取消	商品搜索、商品收藏、交易评价
个人中心	个人信息展示、资料编辑	主题切换、意见反馈、关于页面

二、系统总体设计

2.1 技术架构

本项目采用经典的三层前后端分离架构，各层职责明确，便于开发和维护：

plaintext



2.2 技术栈选型

表格

层级	技术选型	选型理由
前端	UniApp-X (Vue 3 + TypeScript)	跨平台开发框架，使用 UTS 语言，一套代码可编译为 H5、小程序、App 等多个平台
开发工具	HBuilderX / VS Code	uni-app 官方推荐开发工具，集成度高，开发效率高
后端	Spring Boot 3.2 + Java 21	主流企业级 Java 开发框架，生态完善，适合构建 RESTful API 服务
数据库	MySQL 8.0	主流关系型数据库，功能完善，社区活跃，适合课程作业使用
数据访问	Spring Data JPA	简化数据库操作，提供强大的 ORM 功能
跨域处理	Spring CORS	Spring Boot 内置跨域处理，配置简单
构建工具	Maven 3.9+	Java 项目标准构建工具，依赖管理方便
安全加密	Spring Security Crypto	提供 BCryptPasswordEncoder 密码加密功能

2.3 系统整体流程

1. 用户启动 App，系统检查本地存储中是否存在有效的登录凭证
2. 若存在有效凭证，直接进入首页；若不存在，跳转至登录页
3. 用户在登录页输入学号 / 手机号和密码，前端进行表单校验
4. 校验通过后，前端向后端发送登录请求
5. 后端接收请求后，查询数据库验证用户身份
6. 验证通过，后端返回用户基本信息；验证失败，返回错误信息
7. 前端接收登录成功响应，保存用户信息和登录状态，跳转至首页
8. 用户在各功能模块进行操作时，前端通过 HTTP 请求与后端交互
9. 后端接收请求后，根据业务逻辑操作数据库，并将结果返回给前端
10. 前端根据后端返回的结果更新页面展示

三、数据库设计

3.1 数据库基本信息

- 数据库类型：MySQL 8.0
- 数据库名称：icampus
- 字符集：utf8mb4
- 排序规则：utf8mb4_0900_ai_ci
- 数据库用户名：root
- 数据库密码：123456
- 服务器地址：localhost

- 端口：3306

3.2 数据库 ER 图（文字描述）

本项目数据库包含 6 张核心数据表，实体间关系如下：

- 用户（User）与预约（Appointment）：一对多关系，一个用户可以提交多个预约，一个预约只能属于一个用户
- 用户（User）与商品（Goods）：一对多关系，一个用户可以发布多个商品，一个商品只能属于一个用户
- 用户（User）与订单（Order）：一对多关系，一个用户可以购买多个商品创建多个订单，一个订单只能属于一个用户
- 商品（Goods）与订单（Order）：一对多关系，一个商品可以被多次购买创建多个订单，一个订单只能对应一个商品
- 商品（Goods）与分类（Category）：多对一关系，一个商品属于一个分类，一个分类可以包含多个商品
- 公告（Announcement）：独立实体，与其他实体无直接关联

3.3 数据表详细设计

1. 用户表（users）

存储系统用户的基本信息，用于身份认证和个人资料展示。

表格

字段名	数据类型	长度	约束	说明
id	BIGINT	-	主键、自增、非空	用户唯一标识
username	VARCHAR	50	非空、唯一	学号或手机号
password	VARCHAR	100	非空	用户登录密码（BCrypt加密）
nickname	VARCHAR	50	非空	用户昵称
phone	VARCHAR	20	允许空	手机号
email	VARCHAR	100	允许空	邮箱
college	VARCHAR	100	允许空	所在学院
status	INT	-	非空、默认1	用户状态：1-正常，0-禁用
create_time	DATETIME	-	非空、默认当前时间	用户注册时间
update_time	DATETIME	-	自动更新	最后更新时间

注：实际表名为 `users`（复数形式），与 JPA 实体类 `@Table(name = "users")` 注解保持一致。

2. 公告表（announcement）

存储校园各类公告信息，包括教务通知、活动讲座、奖学金通知等。

表格

字段名	数据类型	长度	约束	说明
id	INT	-	主键、自增、非空	公告唯一标识
title	VARCHAR	200	非空	公告标题
content	TEXT	-	非空	公告正文内容
type	VARCHAR	20	非空	公告类型，可选值：教务、活动、奖学金、其他
publish_time	DATETIME	-	非空	公告发布时间
publisher	VARCHAR	50	非空	发布单位，如教务处、学生会、学生处等

3. 服务预约表（appointments）

存储用户提交的各类校园服务预约信息。

表格

字段名	数据类型	长度	约束	说明
id	BIGINT	-	主键、自增、非空	预约唯一标识
user_id	BIGINT	-	非空、外键	关联用户表的 id 字段，标识预约提交人
service_type	VARCHAR	50	非空	服务类型，可选值：自习室预约、宿舍报修、跑腿代取
appointment_time	VARCHAR	50	非空	用户预约的服务时间段（如：14:00-16:00）
location	VARCHAR	200	非空	服务地点，如教学楼 A301、宿舍 1 号楼 203
description	VARCHAR	500	允许空	需求或问题描述
contact	VARCHAR	20	允许空	用户联系方式，手机号
status	VARCHAR	20	非空、默认"待处理"	预约状态：待处理、已通过、已完成、已取消

字段名	数据类型	长度	约束	说明
create_time	DATETIME	-	非空、默认当前时间	预约提交时间
update_time	DATETIME	-	自动更新	最后更新时间

注：实际表名为 `appointments`（复数形式），与 JPA 实体类 `@Table(name = "appointments")` 注解保持一致。description 字段在实体类中定义为 `VARCHAR(500)`。

4. 商品分类表（categories）

存储二手商品的分类信息。

字段名	数据类型	长度	约束	说明
id	INT	-	主键、自增、非空	分类唯一标识
name	VARCHAR	50	非空	分类名称
icon	VARCHAR	50	允许空	分类图标

5. 商品表（goods）

存储二手商品的详细信息。

字段名	数据类型	长度	约束	说明
id	BIGINT	-	主键、自增、非空	商品唯一标识
user_id	BIGINT	-	非空	关联用户表的 id 字段
title	VARCHAR	100	非空	商品标题
description	VARCHAR	500	允许空	商品描述
price	DOUBLE	-	非空	售价
original_price	DOUBLE	-	允许空	原价
category_id	INT	-	允许空	关联分类表的 id 字段
images	VARCHAR	500	允许空	商品图片，JSON 格式存储
status	VARCHAR	20	非空、默认 "出售中"	商品状态：出售中、已售出
contact	VARCHAR	20	允许空	联系方式
location	VARCHAR	200	允许空	交易地点
create_time	DATETIME	-	非空、默认当前时间	发布时间
update_time	DATETIME	-	自动更新	更新时间

注：id 和 user_id 为 BIGINT 类型，与 User 表保持一致。title 字段长度为 100，description 和 images 字段为 VARCHAR(500)。

6. 订单表（orders）

存储二手商品的订单信息。

字段名	数据类型	长度	约束	说明
id	INT	-	主键、自增、非空	订单唯一标识
buyer_id	INT	-	非空	关联用户表的 id 字段
goods_id	INT	-	非空	关联商品表的 id 字段
status	VARCHAR	20	非空、默认 "待确认"	订单状态：待确认、已完成、已取消
create_time	DATETIME	-	非空、默认当前时间	下单时间

四、前端设计

4.1 整体界面设计

本项目采用简洁、清晰的设计风格，主色调为蓝色（#007AFF），代表科技和信任，符合校园应用的定位。界面布局采用卡片式设计，层次分明，便于用户操作。

4.2 导航结构

采用底部 TabBar 导航方式，共设置 3 个主 Tab，分别对应首页、服务和个人中心，用户可通过点击 Tab 快速切换不同功能模块。

4.3 页面清单及功能说明

表格

模块	页面名称	页面功能
账号模块	登录页	用户输入学号 / 手机号和密码进行登录；提供表单校验和错误提示；支持学号/手机号自动识别
首页模块	首页	展示个性化欢迎语；提供公告中心、校园服务、校园二手平台等快捷入口；展示最新 3 条公告预览
公告模块	公告列表页	展示所有公告列表；支持按公告类型筛选；点击公告项可进入详情页
公告模块	公告详情页	展示公告的完整内容，包括标题、发布时间、发布单位和正文
服务模块	服务列表页	展示可预约的校园服务类型，包括自习室预约、宿舍报修、跑腿代取
服务模块	服务预约页	用户填写预约信息并提交；提供表单校验和提交反馈
服务模块	我的预约页	展示用户提交的所有预约记录，按提交时间倒序排列；支持超时自动处理
二手平台	商品列表页	展示所有二手商品；支持按分类筛选；点击商品可进入详情页
二手平台	商品详情页	展示商品完整信息；提供购买按钮；支持创建订单
二手平台	发布商品页	用户填写商品信息并发布；提供表单校验；支持选择分类
二手平台	我的订单页	展示用户的所有订单；支持查看订单状态；支持订单超时自动取消
个人中心	个人主页	展示用户头像、昵称、学号、学院、专业等基本信息；提供编辑资料和退出登录入口
个人中心	编辑资料页	用户修改个人昵称、学院、专业等信息；保存修改并更新个人主页

4.4 核心业务流程设计

4.4.1 用户登录流程

- 1. 输入阶段：**用户在登录页输入学号 / 手机号和密码
- 2. 校验阶段**
 - ：前端对输入内容进行非空校验和格式校验
 - 学号 / 手机号不能为空
 - 密码不能为空，且长度不少于 6 位
- 3. 请求阶段：**校验通过后，前端向后端发送登录请求
- 4. 响应阶段**
 - ：后端接收请求，查询数据库验证用户身份

- 验证成功：返回用户基本信息
 - 验证失败：返回 "用户名或密码错误" 提示
5. 结果处理阶段

:

- 登录成功：前端保存用户信息和登录状态，跳转至首页
- 登录失败：前端显示错误提示信息

4.4.2 公告查看流程

1. 用户在首页点击 "公告中心" 快捷入口或 "最新公告" 右侧的 "全部" 按钮
2. 前端向后端发送获取公告列表的请求
3. 后端查询数据库，返回所有公告数据
4. 前端接收数据，渲染公告列表
5. 用户可点击顶部的类型标签筛选不同类型的公告
6. 用户点击某条公告，前端向后端发送获取该公告详情的请求
7. 后端查询数据库，返回对应公告的详细信息
8. 前端接收数据，渲染公告详情页

4.4.3 服务预约流程

1. 用户在服务列表页选择需要预约的服务类型
2. 跳转至服务预约页，页面标题自动显示为 "XX 预约"
3. 用户填写预约信息，包括预约时间、地点、需求描述和联系方式
4. 前端对输入内容进行表单校验
 - 所有必填项不能为空
 - 需求描述长度不少于 10 字
 - 联系方式格式正确
5. 校验通过后，前端向后端发送提交预约的请求
6. 后端接收请求，将预约信息插入数据库
7. 后端返回提交成功的响应
8. 前端显示 "预约提交成功" 提示，并自动跳转至我的预约页
9. 用户可在我的预约页查看所有预约记录

五、后端接口设计

5.1 接口规范

- 基础 URL: `http://localhost:8080/api`
- 请求方法: GET 用于获取数据, POST 用于提交数据, PUT 用于更新数据, DELETE 用于删除数据
- 请求格式: `application/json`
- 响应格式: `application/json`
- 跨域支持: 所有接口支持跨域访问 (`@CrossOrigin(origins = "*")`)

通用响应结构

```
{
  "success": true,
  "msg": "操作成功",
  "data": {}
}
```

响应字段说明

字段名	类型	说明
success	Boolean	操作是否成功，true 表示成功，false 表示失败
msg	String	操作结果消息，成功时返回成功提示，失败时返回错误原因
data	Object	响应数据，查询操作返回数据对象或数组，其他操作可能为 null

HTTP 状态码说明

状态码	说明
200	操作成功
400	请求参数错误
401	身份验证失败
404	资源不存在
500	服务器内部错误

命名规范

- **URL 路径**: 使用小写字母，单词间用连字符 (-) 分隔，如 /api/secondhand/goods
- **请求参数**: 使用驼峰命名法，如 userId、serviceType
- **响应字段**: 使用驼峰命名法，如 createTime、updateTime
- **路径参数**: 使用 {id} 格式，如 /api/users/{id}

5.2 核心接口详细设计

1. 用户登录接口

- 接口地址: /api/users/login
- 请求方法: POST
- 请求参数:

表格

参数名	类型	必选	说明
username	String	是	学号或手机号
password	String	是	登录密码

- 响应示例（成功）：

```
{
  "success": true,
  "msg": "登录成功",
  "data": {
    "id": 1,
    "username": "2026001",
    "nickname": "测试学生",
    "phone": "13800138001",
    "email": "test@example.com",
    "college": "计算机学院"
  }
}
```

- 响应示例（失败）：

```
{
  "success": false,
  "msg": "用户名或密码错误",
  "data": null
}
```

1.1 用户注册接口

- 接口地址： `/api/users/register`
- 请求方法：POST
- 请求参数：

参数名	类型	必选	说明
username	String	是	学号或手机号
password	String	是	登录密码
nickname	String	是	用户昵称

- 响应示例（成功）：

```
{
  "success": true,
  "msg": "注册成功",
  "data": {
    "id": 1,
    "username": "2026001",
    "nickname": "测试学生"
  }
}
```

2. 获取公告列表接口

- 接口地址： `/announcements`
- 请求方法：GET
- 请求参数：无
- 响应示例：

json

```
{
  "success": true,
  "msg": "获取成功",
  "data": [
    {
      "id": 1,
      "title": "关于期末考试安排的通知",
      "type": "教务",
      "publish_time": "2026-04-25 10:00:00",
      "publisher": "教务处"
    },
    {
      "id": 2,
      "title": "校园读书分享会报名",
      "type": "活动",
      "publish_time": "2026-04-26 14:00:00",
      "publisher": "学生会"
    }
  ]
}
```

3. 获取公告详情接口

- 接口地址: /announcements/:id
- 请求方法: GET
- 请求参数:

表格

参数名	类型	必选	说明
id	Number	是	公告 ID

- 响应示例:

json

```
{
  "success": true,
  "msg": "获取成功",
  "data": {
    "id": 1,
    "title": "关于期末考试安排的通知",
    "content": "期末考试将于第18周进行, 请同学们做好复习准备。",
    "type": "教务",
    "publish_time": "2026-04-25 10:00:00",
    "publisher": "教务处"
  }
}
```

4. 提交服务预约接口

- 接口地址: /api/appointments
- 请求方法: POST

- 请求参数:

表格

参数名	类型	必选	说明
userId	Number	是	用户 ID
serviceType	String	是	服务类型
appointmentTime	String	是	预约时间, 格式: 14:00-16:00
location	String	是	服务地点
description	String	否	需求描述
contact	String	否	联系方式

- 响应示例:

```
{
  "success": true,
  "msg": "创建成功",
  "data": {
    "id": 1,
    "userId": 1,
    "serviceType": "宿舍报修",
    "appointmentTime": "14:00-16:00",
    "location": "宿舍1号楼203",
    "description": "水龙头漏水",
    "contact": "13800138001",
    "status": "待处理",
    "createTime": "2026-05-13T10:00:00",
    "updateTime": "2026-05-13T10:00:00"
  }
}
```

5. 获取我的预约接口

- 接口地址: /api/appointments
- 请求方法: GET
- 请求参数:

表格

参数名	类型	必选	说明
user_id	Number	否	用户 ID, 查询指定用户的预约

- 响应示例:

```
{
  "success": true,
  "msg": "获取成功",
  "data": [
    {
      "id": 1,
```

```
    "userId": 1,
    "serviceType": "宿舍报修",
    "appointmentTime": "14:00-16:00",
    "location": "宿舍1号楼203",
    "description": "水龙头漏水",
    "contact": "13800138001",
    "status": "待处理",
    "createTime": "2026-05-13T10:00:00",
    "updateTime": "2026-05-13T10:00:00"
  }
]
```

5.1 更新预约状态接口

- 接口地址: `/api/appointments/{id}/status`
- 请求方法: PUT
- 请求参数:

参数名	类型	必选	说明
id	Number	是	预约 ID (路径参数)
status	String	是	新状态 (请求体)

- 响应示例:

```
{
  "success": true,
  "msg": "状态更新成功",
  "data": null
}
```

5.2 删除预约接口

- 接口地址: `/api/appointments/{id}`
- 请求方法: DELETE
- 请求参数:

参数名	类型	必选	说明
id	Number	是	预约 ID (路径参数)

- 响应示例:

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

6. 获取用户信息接口

- 接口地址: `/api/users/{id}`

- 请求方法: GET
- 请求参数:

参数名	类型	必选	说明
id	Number	是	用户 ID (路径参数)

- 响应示例:

```
{
  "success": true,
  "msg": "获取成功",
  "data": {
    "id": 1,
    "username": "2026001",
    "nickname": "测试学生",
    "phone": "13800138001",
    "email": "test@example.com",
    "college": "计算机学院",
    "status": 1,
    "createTime": "2026-05-13T10:00:00",
    "updateTime": "2026-05-13T10:00:00"
  }
}
```

6.1 更新用户信息接口

- 接口地址: /api/users/{id}
- 请求方法: PUT
- 请求参数:

参数名	类型	必选	说明
id	Number	是	用户 ID (路径参数)
nickname	String	否	用户昵称
phone	String	否	手机号
email	String	否	邮箱
college	String	否	学院

- 响应示例:

```
{
  "success": true,
  "msg": "更新成功",
  "data": {
    "id": 1,
    "username": "2026001",
    "nickname": "新昵称",
    "phone": "13800138001",
    "email": "test@example.com",
    "college": "计算机学院",
    "status": 1
  }
}
```

6.2 删除用户接口

- 接口地址: `/api/users/{id}`
- 请求方法: DELETE
- 请求参数:

参数名	类型	必选	说明
id	Number	是	用户 ID (路径参数)

- 响应示例:

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

7. 获取商品分类接口

- 接口地址: `/secondhand/categories`
- 请求方法: GET
- 请求参数: 无
- 响应示例:

```
{
  "success": true,
  "msg": "获取成功",
  "data": [
    {
      "id": 1,
      "name": "书籍教材",
      "icon": "📖"
    },
    {
      "id": 2,
      "name": "电子产品",
      "icon": "📱"
    }
  ]
}
```



```
]
}
```

8. 获取商品列表接口

- 接口地址: `/api/secondhand/goods`
- 请求方法: GET
- 请求参数:

参数名	类型	必选	说明
user_id	Number	否	用户 ID
category_id	Number	否	分类 ID
keyword	String	否	搜索关键词

- 响应示例:

```
{
  "success": true,
  "msg": "获取成功",
  "data": [
    {
      "id": 1,
      "userId": 1,
      "title": "二手笔记本电脑",
      "description": "使用两年, 性能良好",
      "price": 2500,
      "originalPrice": 5000,
      "categoryId": 2,
      "images": "[\"image1.jpg\"]",
      "status": "出售中",
      "contact": "13800138001",
      "location": "计算机学院",
      "createTime": "2026-05-13T10:00:00",
      "updateTime": "2026-05-13T10:00:00"
    }
  ]
}
```

9. 获取商品详情接口

- 接口地址: `/api/secondhand/goods/{id}`
- 请求方法: GET
- 请求参数:

参数名	类型	必选	说明
id	Number	是	商品 ID (路径参数)

- 响应示例:

```
{
  "success": true,
```

```
"msg": "获取成功",
"data": {
  "id": 1,
  "userId": 1,
  "title": "二手笔记本电脑",
  "description": "使用两年, 性能良好",
  "price": 2500,
  "originalPrice": 5000,
  "categoryId": 2,
  "images": "[\"image1.jpg\"]",
  "status": "出售中",
  "contact": "13800138001",
  "location": "计算机学院",
  "createTime": "2026-05-13T10:00:00",
  "updateTime": "2026-05-13T10:00:00"
}
```

10. 发布商品接口

- 接口地址: `/api/secondhand/goods`
- 请求方法: POST
- 请求参数:

参数名	类型	必选	说明
userId	Number	是	用户 ID
title	String	是	商品标题
price	Number	是	售价
description	String	否	商品描述
originalPrice	Number	否	原价
categoryId	Number	否	分类 ID
images	String	否	商品图片, JSON 格式
contact	String	否	联系方式
location	String	否	交易地点

- 响应示例:

```
{
  "success": true,
  "msg": "发布成功",
  "data": {
    "id": 1,
    "userId": 1,
    "title": "二手笔记本电脑",
    "description": "使用两年, 性能良好",
    "price": 2500,
    "originalPrice": 5000,
    "categoryId": 2,
```

```
    "images": "[\"image1.jpg\"]",
    "status": "出售中",
    "contact": "13800138001",
    "location": "计算机学院",
    "createTime": "2026-05-13T10:00:00",
    "updateTime": "2026-05-13T10:00:00"
  }
}
```

10.1 更新商品接口

- 接口地址: `/api/secondhand/goods/{id}`
- 请求方法: PUT
- 请求参数:

参数名	类型	必选	说明
id	Number	是	商品 ID (路径参数)
title	String	否	商品标题
price	Number	否	售价
description	String	否	商品描述
originalPrice	Number	否	原价
categoryId	Number	否	分类 ID
images	String	否	商品图片
status	String	否	商品状态
contact	String	否	联系方式
location	String	否	交易地点

- 响应示例:

```
{
  "success": true,
  "msg": "更新成功",
  "data": { ... }
}
```

10.2 删除商品接口

- 接口地址: `/api/secondhand/goods/{id}`
- 请求方法: DELETE
- 请求参数:

参数名	类型	必选	说明
id	Number	是	商品 ID (路径参数)

- 响应示例:

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

11. 创建订单接口

- 接口地址: `/secondhand/orders`
- 请求方法: POST
- 请求参数:

参数名	类型	必选	说明
buyer_id	Number	是	买家 ID
goods_id	Number	是	商品 ID

- 响应示例:

```
{
  "success": true,
  "msg": "下单成功",
  "data": { "id": 1 }
}
```

12. 获取我的订单接口

- 接口地址: `/secondhand/my-orders/:user_id`
- 请求方法: GET
- 请求参数:

参数名	类型	必选	说明
user_id	Number	是	用户 ID

- 响应示例:

```
{
  "success": true,
  "msg": "获取成功",
  "data": [
    {
      "id": 1,
      "goods_id": 1,
      "title": "二手笔记本电脑",
      "price": 2500,
      "status": "待确认",
      "create_time": "2026-04-28 10:00:00"
    }
  ]
}
```

13. 更新订单状态接口

- 接口地址: `/secondhand/orders/:id`
- 请求方法: PUT
- 请求参数:

参数名	类型	必选	说明
id	Number	是	订单 ID
status	String	是	订单状态

- 响应示例:

```
{
  "success": true,
  "msg": "更新成功",
  "data": null
}
```

六、项目目录结构

6.1 后端项目结构 (springboot-backend)

```
springboot-backend/
├── pom.xml                                # Maven 项目配置文件
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── example/
│   │   │   │   │   ├── icampus/
│   │   │   │   │   │   ├── IcampusApplication.java    # 应用启动类
│   │   │   │   │   │   ├── common/
│   │   │   │   │   │   │   ├── Response.java          # 统一响应封装
│   │   │   │   │   │   │   ├── config/
│   │   │   │   │   │   │   │   ├── webConfig.java      # web 配置（跨域等）
│   │   │   │   │   │   │   ├── controller/             # 控制器层
│   │   │   │   │   │   │   │   ├── UserController.java
│   │   │   │   │   │   │   │   ├── AppointmentController.java
│   │   │   │   │   │   │   │   ├── GoodsController.java
│   │   │   │   │   │   │   ├── entity/                 # 实体类
│   │   │   │   │   │   │   │   ├── User.java
│   │   │   │   │   │   │   │   ├── Appointment.java
│   │   │   │   │   │   │   │   ├── Goods.java
│   │   │   │   │   │   │   ├── repository/              # 数据访问层
│   │   │   │   │   │   │   │   ├── UserRepository.java
│   │   │   │   │   │   │   │   ├── AppointmentRepository.java
│   │   │   │   │   │   │   │   ├── GoodsRepository.java
│   │   │   │   │   │   │   ├── service/                 # 业务逻辑层
│   │   │   │   │   │   │   │   ├── UserService.java
│   │   │   │   │   │   │   │   ├── AppointmentService.java
│   │   │   │   │   │   │   │   ├── GoodsService.java
│   │   │   │   │   │   │   │   └── impl/                # 实现类
```


- 开发工具：HBuilderX 3.8.12 / VS Code
- 后端运行环境：Java 21 + Spring Boot 3.2
- 数据库：MySQL 8.0
- 数据库管理工具：Navicat Premium 16 / MySQL Workbench
- 浏览器：Google Chrome 124.0.0.0
- 前端构建工具：Vite 5.4.21
- 后端构建工具：Maven 3.9+
- 包管理工具：npm / Maven
- 后端服务端口：8080
- 前端服务端口：8083

7.2 启动说明

后端启动

1. 确保 MySQL 数据库已启动并创建 `icampus` 数据库
2. 进入 `springboot-backend` 目录
3. 执行 `mvn spring-boot:run` 或运行 `IcampusApplication.java`
4. 服务将在 `http://localhost:8080` 启动

前端启动

1. 进入 `uniappx-frontend` 目录
2. 执行 `npm install` 安装依赖
3. 执行 `npm run dev:h5` 启动 H5 开发服务器
4. 访问 `http://localhost:8083` 查看应用

7.3 开发进度

表格

阶段	时间	完成内容
需求分析与设计	第 1 周	完成需求分析、系统设计、数据库设计和接口设计
数据库初始化	第 2 周	创建数据库和数据表，插入测试数据
后端开发	第 2-3 周	完成数据库连接和所有核心接口的开发
前端开发	第 3-4 周	完成所有页面的 UI 开发和交互逻辑
前后端联调	第 5 周	对接前后端接口，修复联调过程中发现的问题
测试与优化	第 6 周	进行功能测试和界面优化，完成项目

7.4 测试方案

7.4.1 功能测试

对所有核心功能进行测试，确保每个功能都能正常运行，测试用例如下：

表格

测试模块	测试用例	预期结果	测试结果
登录模块	输入正确的账号和密码	登录成功，跳转至首页	通过
登录模块	输入正确的账号密码	登录成功，跳转至首页	通过
登录模块	输入错误的密码	提示 "用户名或密码错误"	通过
登录模块	账号为空	提示 "请输入学号或手机号"	通过
登录模块	密码为空	提示 "请输入密码"	通过
登录模块	输入手机号码	自动识别为手机号输入	通过
首页模块	登录成功进入首页	显示正确的欢迎语和最新公告	通过
公告模块	进入公告列表页	显示所有公告，按发布时间倒序排列	通过
公告模块	点击 "教务" 标签	只显示教务类型的公告	通过
公告模块	点击公告列表项	跳转至公告详情页，显示完整内容	通过
服务模块	点击 "宿舍报修"	跳转至宿舍报修预约页	通过
服务模块	提交完整的预约信息	预约提交成功，跳转至我的预约页	通过
服务模块	进入我的预约页	显示用户提交的所有预约记录	通过
二手平台	进入商品列表页	显示所有二手商品	通过
二手平台	点击分类标签	只显示对应分类的商品	通过
二手平台	点击商品列表项	跳转至商品详情页	通过
二手平台	发布商品（已登录）	商品发布成功	通过
二手平台	发布商品（未登录）	提示 "请先登录"	通过
二手平台	购买商品（已登录）	创建订单成功	通过
二手平台	购买商品（未登录）	提示 "请先登录"	通过
二手平台	进入我的订单页	显示用户的所有订单	通过
个人中心	进入个人主页	显示正确的用户信息	通过
个人中心	修改昵称并保存	个人主页显示更新后的昵称	通过
个人中心	点击 "退出登录"	清除登录状态，跳转至登录页	通过

7.4.2 数据库测试

1. 插入测试数据，验证数据是否正确存入数据库
2. 查询数据，验证返回结果是否与数据库中的数据一致
3. 更新数据，验证数据库中的数据是否被正确修改
4. 删除数据，验证数据库中的数据是否被正确删除

7.4.3 兼容性测试

在 Google Chrome、Microsoft Edge 等主流浏览器中测试项目，确保界面显示正常，功能运行稳定。

7.5 测试结果

所有测试用例均通过，项目功能完整，运行稳定，无明显 bug，达到了课程作业的要求。

八、项目总结

8.1 完成情况

本项目按照课程要求，完成了一个基于 UniApp-X + Spring Boot + MySQL 的智慧校园生活 App，实现了用户登录、公告查看、服务预约、校园二手交易、个人信息管理等核心功能。项目采用前后端分离架构：

- **前端**：UniApp-X 框架 (Vue 3 + TypeScript + UTS)，支持 H5 端运行
- **后端**：Spring Boot 3.2 + Java 21，提供 RESTful API 服务
- **数据库**：MySQL 8.0，数据持久化存储

代码结构清晰，注释规范，界面美观，交互流畅，达到了合格偏上的课程作业水平。

主要完成功能：

- **账号模块**：学号/手机号登录、用户注册、退出登录、登录状态保持
- **首页模块**：个性化欢迎语、快捷功能入口、最新公告预览、二手商品展示
- **公告模块**：公告列表展示、公告详情查看
- **服务模块**：服务类型列表、提交预约申请、我的预约查询、预约状态管理、时间沙漏倒计时
- **二手平台**：商品列表展示、商品分类筛选、商品详情、发布商品、联系卖家
- **个人中心**：个人信息展示、资料编辑

8.2 遇到的问题与解决方法

1. **前后端跨域问题**：在开发过程中，前端请求后端接口时出现跨域错误。
 - **解决方法**：在后端 Controller 类上添加 `@CrossOrigin(origins = "*")` 注解，允许跨域访问。
2. **API 接口路径不一致**：前端请求路径与后端定义的路径不匹配，导致 404 错误。
 - **解决方法**：统一接口路径规范，所有接口以 `/api` 开头，如 `/api/users/login`。
3. **数据库表名与实体类不一致**：JPA 实体类默认使用复数表名，与文档中的单数表名不一致。
 - **解决方法**：使用 `@Table(name = "users")` 注解明确指定表名，并在文档中同步更新。
4. **请求参数命名规范**：前端使用驼峰命名 (`userId`)，但文档中描述为下划线命名 (`user_id`)。
 - **解决方法**：统一使用驼峰命名法，符合 Java 和 JavaScript 的命名习惯。
5. **Lombok 注解处理器问题**：编译时出现 Lombok 相关错误。
 - **解决方法**：在 `pom.xml` 中配置 Maven 编译器插件，添加 Lombok 注解处理器路径。
6. **图片上传 Blob URL 问题**：前端使用临时 Blob URL 存储图片，导致数据持久化失败。
 - **解决方法**：清理数据库中的无效 Blob URL，修改图片上传逻辑，确保只保存有效的图片地址。
7. **联系卖家功能跨平台兼容**：`uni.makePhoneCall` 在 H5 环境下不支持。
 - **解决方法**：添加平台判断，H5 环境使用 HTML `<a>` 标签配合 `tel:` 协议实现拨打电话功能。

8.3 技术亮点

1. **前后端分离架构**：前端和后端独立开发，通过 RESTful API 通信，便于维护和扩展。
2. **统一响应封装**：后端使用统一的 `Response` 类封装响应结果，前端处理逻辑更加清晰。

- 3. **JPA 数据访问**: 使用 Spring Data JPA 简化数据库操作, 自动生成 CRUD 方法。
- 4. **跨域处理**: 使用 Spring CORS 注解实现跨域, 配置简单高效。
- 5. **密码加密存储**: 使用 BCryptPasswordEncoder 对用户密码进行加密, 保障数据安全。
- 6. **时间自动管理**: 使用 JPA 的 @PrePersist 和 @PreUpdate 注解自动管理创建时间和更新时间。

8.4 改进方向

- 1. **用户认证**: 引入 JWT 令牌机制, 实现更安全的用户认证和授权。
- 2. **图片上传**: 集成云存储服务 (如阿里云 OSS、七牛云), 实现真正的图片上传功能。
- 3. **订单系统**: 完善二手平台的订单管理功能, 包括订单状态流转、支付集成等。
- 4. **消息通知**: 添加系统消息推送功能, 及时通知用户预约状态变更、订单消息等。
- 5. **数据分页**: 对列表查询接口添加分页支持, 提升大数据量下的查询性能。
- 6. **缓存优化**: 引入 Redis 缓存热点数据, 减轻数据库压力。
- 7. **单元测试**: 完善后端单元测试, 提高代码质量和可维护性。

8.5 开发过程中的问题记录

序号	问题描述	解决方法
1	前后端跨域问题	在后端 Controller 类上添加 @CrossOrigin(origins = "*") 注解
2	API 接口路径不一致	统一接口路径规范, 所有接口以 /api 开头
3	数据库表名与实体类不一致	使用 @Table(name = "users") 注解明确指定表名
4	请求参数命名规范	统一使用驼峰命名法 (userId)
5	Lombok 注解处理器问题	在 pom.xml 中配置 Maven 编译器插件, 添加 Lombok 注解处理器路径
6	图片上传 Blob URL 问题	清理数据库中的无效 Blob URL, 修改图片上传逻辑
7	联系卖家功能跨平台兼容	H5 环境使用 HTML <a> 标签配合 tel: 协议实现拨打电话
8	登录状态管理问题	统一使用 getUser() 函数获取用户信息
9	TabBar 图标加载中断错误	开发环境正常现象, 生产环境不会出现
10	数据库连接失败	检查数据库服务是否启动, 确认连接信息是否正确

8.6 收获与体会

通过本次课程作业, 我系统地学习了跨平台移动应用开发的全过程:

- 1. **前端开发**: 掌握了 UniApp-X 框架的基本使用方法, 理解了 Vue 3 + TypeScript 的开发模式, 学会了使用 UTS 语言进行跨平台开发。
- 2. **后端开发**: 学会了使用 Spring Boot 3.2 + Java 21 构建 RESTful API 服务, 理解了分层架构 (Controller-Service-Repository) 的设计思想。
- 3. **数据库设计**: 掌握了 MySQL 数据库的设计方法, 理解了关系型数据库的表结构设计原则。

4. **前后端联调**: 学会了前后端分离项目的开发和联调方法, 理解了 HTTP 协议和 RESTful API 设计规范。
5. **软件工程**: 认识到了软件工程的重要性, 一个好的项目需要经过详细的需求分析、系统设计、编码实现和测试优化等多个阶段。

8.7 未来改进方向

本项目目前仅实现了基础功能, 未来可以从以下几个方面进行改进和扩展:

1. **用户认证**: 引入 JWT 令牌机制, 实现更安全的用户认证和授权
2. **图片上传**: 集成云存储服务 (阿里云 OSS、七牛云), 实现真正的图片上传功能
3. **订单系统**: 完善二手平台的订单管理功能, 包括订单状态流转、支付集成等
4. **消息通知**: 添加系统消息推送功能, 及时通知用户预约状态变更、订单消息等
5. **数据分页**: 对列表查询接口添加分页支持, 提升大数据量下的查询性能
6. **缓存优化**: 引入 Redis 缓存热点数据, 减轻数据库压力
7. **单元测试**: 完善后端单元测试, 提高代码质量和可维护性
8. **部署上线**: 部署到云服务器, 实现真正的线上访问

八、交付物清单

1. 完整的项目源代码 (前端 + 后端)
2. 数据库 SQL 脚本 (包含表结构和测试数据)
3. 本开发文档
4. 项目运行说明文档
5. 项目演示截图